GÜMÜŞHANE ÜNİVERSİTESİ
İKTİSADİ ve İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİTEMLERİ BÖLÜMÜ

# YBS 207 Algoritma ve Programlamaya Giriş

## String Sınıfı ve Metotları

Öğr. Gör. Dr. Baki GÖKGÖZ

# Giriş

- **String Tanımı:**
  - **Stringler,** kelimeleri, cümleleri ve paragrafları oluşturmak için kullanılan alfabe (karakter) koleksiyonlarıdır.
  - **Stringler,** programlamada en sık kullanılan veri türleri arasındadır.

- **Python String Sınıfı:**
  - Python, stringleri işlemek için `str` adlı bir sınıf (tür) sağlar.
  - Stringler, Unicode karakterlerinin değişmez dizileridir.
  - Stringler çeşitli operatörleri ve yöntemleri destekler.

- **String Operatörleri:**
  - **Stringlerde uygulanabilen yaygın operatörler:**
    - concatenation (`+`),
    - repetition (`*`),
    - membership (`in`, `not in`) ve
    - indexing/slicing bulunur.

# Giriş

- **String Metotları:**
  - Metotlar, arama, değiştirme, bölme, biçimlendirme ve daha fazlası gibi işlemleri gerçekleştirmek için string nesneleriyle ilişkili fonksiyonlardır.
  - Metotlar, yaygın string işlemlerini basitleştirir ve kod okunabilirliğini artırır.
    - **`upper()`** → Tüm karakterleri BÜYÜK harfe çevirir.
    - **`lower()`** → Tüm karakterleri küçük harfe çevirir.
    - **`replace("eski", "yeni")`** → Belirtilen kısmı değiştirir.
    - **`split(" ")`** → String'i belirtilen ayraça göre böler (liste döndürür).
    - **`strip()`** → Baştaki ve sondaki boşlukları siler.
    - **`find("aranan")`** → Bir ifadenin konumunu bulur.

  - Python kodları:

```
text = " Merhaba Dünya "
print(text.upper())        # " MERHABA DÜNYA "
print(text.strip())        # "Merhaba Dünya"
print(text.find("Dünya")) # 10
print(text.replace("Dünya", "Python")) # "Merhaba Python"
```

# String Nedir

- **String (Dize) Nedir?**
  - Python'da bir string, tırnak işaretleri **('...' veya "...")** içine alınmış karakterler topluluğudur.
  - Büyük ve küçük harfler, sayılar, özel karakterler **(`@, ., vb.`)**, boşluklar veya herhangi bir klavye karakterini içerebilir.

```
s1='Merhaba Dünya'

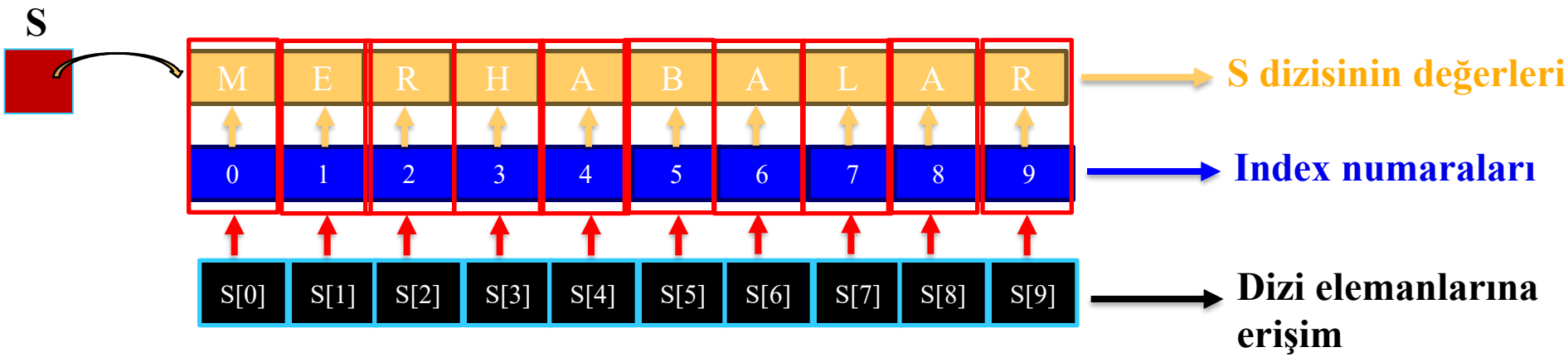
s2='123548866'

s3='ab3.785-?/20'
```

  - Stringler metin verilerini temsil eder ve programlamada temel bir veri tipidir.

# Python'da String Gösterimi

- **Python'da String Gösterimi**
  - İçerik olarak, bir **string** her bir karakterin bir konum (indeks) kapladığı bir dizi olarak saklanır.
  - İndeksler ilk karakter için **0**'dan başlar ve sonraki her bir karakter için **1** artar
  - Karakterlere erişim bu indeksler kullanılarak yapılır,

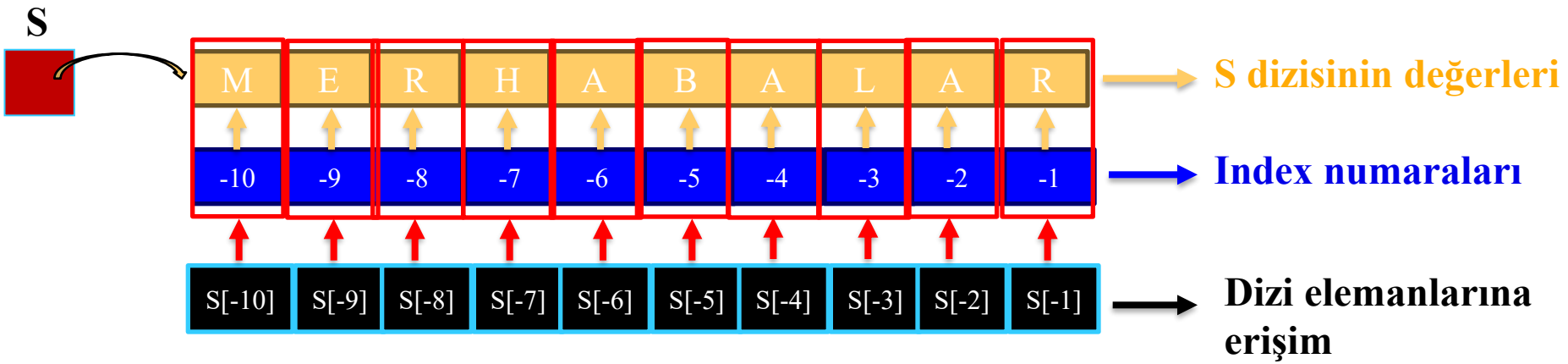



```
print(S[0])
```
→ M

```
print(S[5])
```
→ B

# Python'da String Gösterimi

- **Python'da String Gösterimi**
  - Python ayrıca negatif indeksleri de destekler; son karakter -1'den başlayarak geriye doğru gider (örneğin, `R=-1, A=-2, ...`) .




  - Karakterlere erişim bu indeksler kullanılarak yapılır, örneğin **`s1[0]`** veya **s1[-1]** sırasıyla ilk ve son karakteri döndürür.

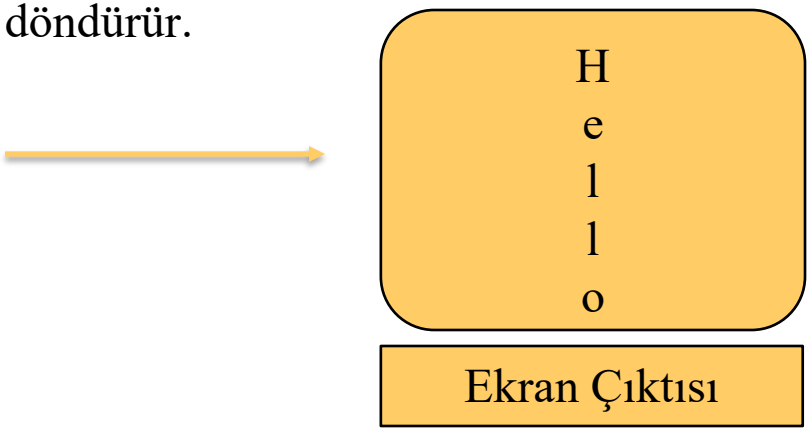
```
print(S[-1])
```
→ R

```
print(S[-3])
```
→ L

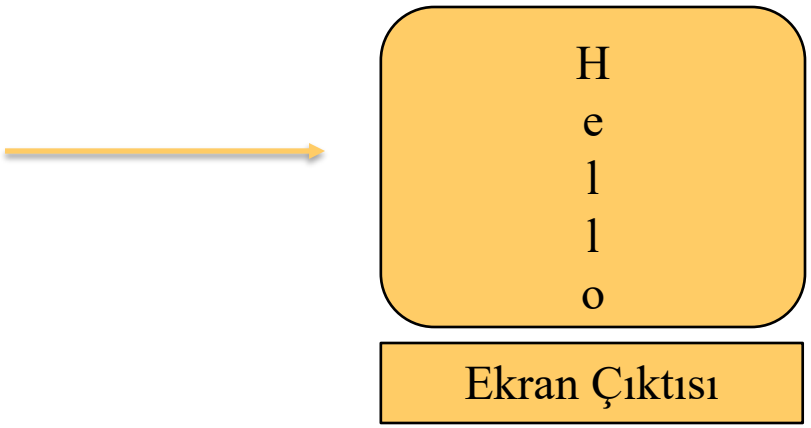
# String Uzunluğunu Ölçme

- **String Uzunluğunu Ölçme**
  - Bir string'in uzunluğu, **boşluklar** ve **özel karakterler** dahil olmak üzere içerdiği toplam karakter sayısıdır.
  - **len()** fonksiyonu, string'ler de dahil olmak üzere herhangi bir dizinin uzunluğunu döndürür.
    - **Örnek:** len("Hello") 7 değerini döndürür.

```
s1 = 'Hello'
for x in s1:
    print(x)
```




```
s2 = 'Hello'
for in in range(len(s2)):
    print(s2[i])
```



# String Literalları

- **String Literalları**
  - String'ler şu şekillerde tanımlanabilir:
    - Tek tırnak: `'Merhaba'`
    - Çift tırnak: `"Merhaba"`
    - Üçlü tek tırnak: `'''Merhaba'''` (çok satırlı string'ler için kullanışlıdır)
    - Üçlü çift tırnak: `"""Merhaba"""` (aynı zamanda çok satırlı string'leri destekler)
    - Üçlü tırnaklar çok satırlı metinler veya açıklamalar (docstring'ler) için kullanılabilir.
  - String'in içinde kesme işareti veya tek tırnak kullanmak için, string çift tırnak içine alınır: `"Ali'nin kalemi"`.
  - Benzer şekilde, string'in içinde çift tırnak kullanmak için, string tek tırnak içine alınır: `'Onlara "Merhaba" dedi'`.

# Index Dilimleme

- **String Indeksleme**
  - String'ler, indeksler aracılığıyla erişilebilen karakter dizileridir.
  - Indeksler ilk karakter için **0'**dan başlar (soldan sağa).
  - Negatif indeksler son karakter için **-1'**den başlar (sağdan sola).
  - Erişim sözdizimi:
    - `string[indeks]`
      - indeks pozitif veya negatif olabilir.
  - Örnek:
    - `s1='Merhaba Dünya'`
      - string'inde `s[0]` bize 'M' değerini verir.
  - String'ler değiştirilemez (immutable):
    - karakterler doğrudan değiştirilemez
    - Örnek:
      - `s[0] = 'N'` **geçersizdir.**
  - Bir string oluşturulduktan sonra, karakterleri ve uzunluğu değiştirilemez.
  - Herhangi bir "değişiklik" işlemi aslında yeni bir string nesnesi oluşturur.

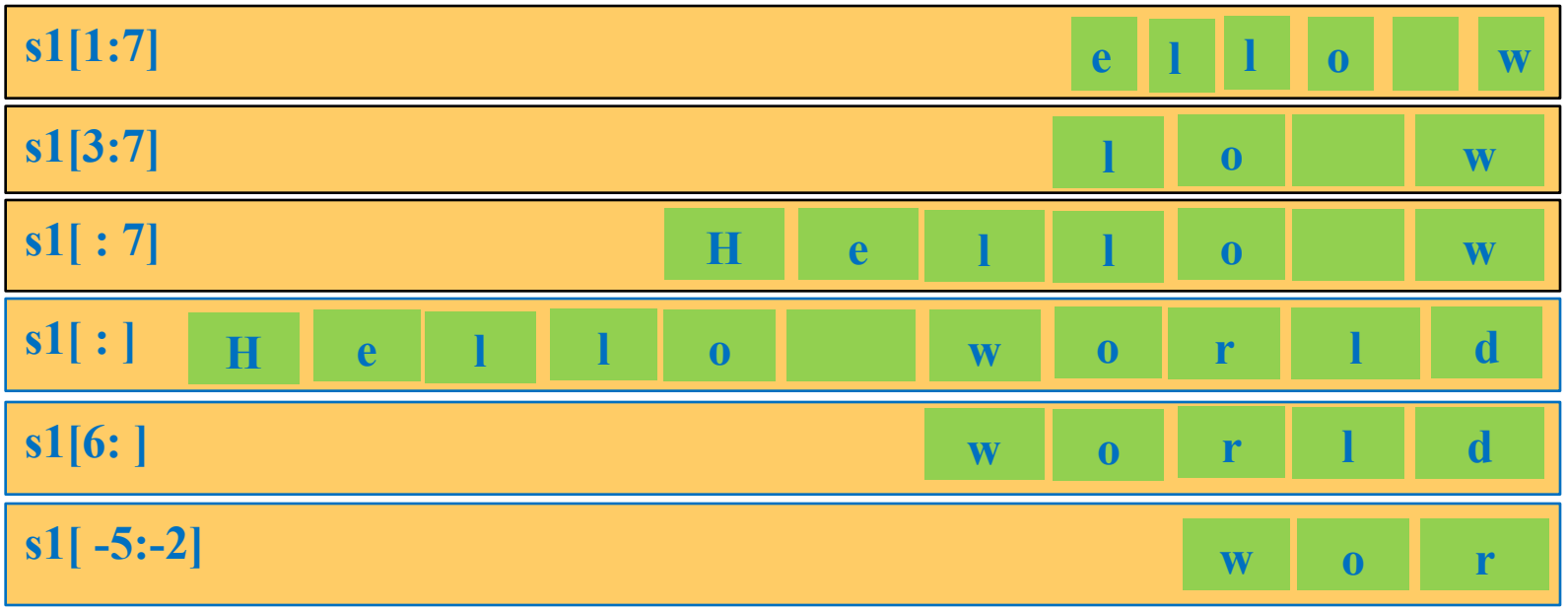
# String Indexleme

- **String Indexleme Syntax:**

range [ start: stop: step ]

  - Parametreler:
    - start: İsteğe bağlı, varsayılan = 0.
    - stop: Zorunlu, döngü bu değere kadar çalışır ancak bu değeri hariç tutar.
    - step: İsteğe bağlı, varsayılan = 1, yani artış değeri.
  - Python kodları:

```
s1 = "Hello world"
```



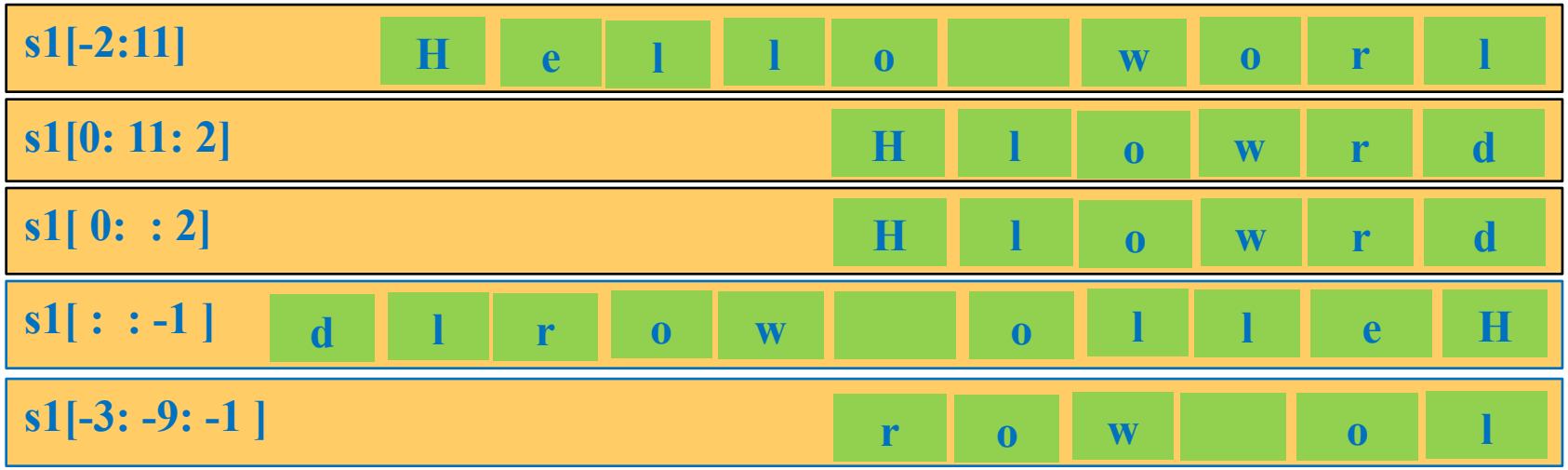
# String Indexleme

- **String Indexleme Syntax:**

range [ start: stop: step ]

– Python kodları:

```
s1 = "Hello world"
```



# String İşlemleri

- **String Birleştirme (Concatenation)**
  - (+) operatörü kullanılarak string'ler birleştirilebilir
  - Python kodu

```
# Temel birleştirme
string1 = "Merhaba"
string2 = "Dünya"
birlesik = string1 + " " + string2
print(birlesik)  # 'Merhaba Dünya'
```

- **String Tekrarlama (Repetition)**
  - (*) operatörü kullanılarak bir string belirli sayıda tekrarlanabilir
  - Python kodu

```
x = 'abc'
y =x*3
print(y)    # abcabcabc
```

# find() ve index() Metotları

- **Alt String Bulma Metotları**
  - **find()**
  - `find(sub, start=0, end=len(string))`
  - String içinde `sub` string'ini arar
  - `sub` string'inin bulunduğu en düşük indeksi döndürür
- **Özellikler:**
  - İsteğe bağlı `start` ve `end` parametreleri, aranacak string dilimini belirtir
  - `sub` string'i bulunamazsa `-1` döndürür
  - Büyük/küçük harf duyarlıdır

# find() ve index() Metotları

- **Özellikler:**
  - İsteğe bağlı `start` ve `end` parametreleri, aranacak string dilimini belirtir
  - `sub` string'i bulunamazsa `-1` döndürür
  - Büyük/küçük harf duyarlıdır
- **Örnekler:**

```
s = "Merhaba Dünya"
print(s.find('a'))          # 4 (ilk 'a' karakterinin indeksi)
print(s.find('Dünya'))      # 8 ('Dünya' alt string'inin başlangıç indeksi)
print(s.find('x'))          # -1 (bulunamadı)
```

  - **Başlangıç parametresi ile**

```
print(s.find('a', 5))       # 6 (5. indeksten sonraki ilk 'a')
print(s.find('a', 7))       # -1 (7. indeksten sonra 'a' yok)
```

  - **Başlangıç ve bitiş parametreleri ile**

```
print(s.find('a', 0, 5))    # 4 (0-5 aralığındaki ilk 'a')
print(s.find('a', 5, 7))    # 6 (5-7 aralığındaki ilk 'a')
```

  - **Kelime arama**

```
print(s.find('haba'))       # 3 ('haba' alt string'inin başlangıcı)
```

# rfind() ve count() Metotları

- **Benzer Metotlar:**
- **Örnekler:**

```
s = "Merhaba Dünya"
```

- `rfind()` - sağdan sola arama (son bulunanın indeksi)

```
print(s.rfind('a'))        # 12 (sondaki 'a' karakterinin indeksi)
```

- `index()` - `find()` gibi çalışır ama bulamazsa hata verir

```
print(s.index('a'))        # 4
print(s.index('x'))        # ValueError: substring not found
```

- `count()` - alt string'in kaç kez geçtiğini sayar

```
print(s.count('a'))          # 3 (3 tane 'a' var)
```